

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1142

COURSE OUTCOME COVERED

CO4:

implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (BL5)

Assessment Tool Weightage: 30%

QUESTIONS: 1

Total Marks:10

Annexure A

Reverse Engineering Task

Code of Conduct:

I Annajothi S (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Annajothi S

Annexure A

Reverse Engineering Task

6.A legacy system tightly binds object creation logic within multiple classes, making it difficult to manage dependencies and extend functionality.

Question:

Reverse engineer the system by applying appropriate creational design patterns (such as Factory Method or Abstract Factory) to decouple object creation from usage. Explain how your redesign improves scalability, flexibility, and code reusability.

RUBRICS FOR CALCULATION

Reverse Engineering Task

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

Reverse Engineering a Legacy System Using Factory Method and Abstract Factory Design Patterns (Elaborate Answer)

Introduction

Legacy software systems are often developed without proper separation of concerns, causing business logic and object creation logic to become tightly coupled. In such systems, classes create objects directly using constructors (for example, `new Doctor()` or `new Patient()`), making the software difficult to maintain and extend. As the application grows, developers must modify multiple classes whenever a new object type is introduced, increasing the risk of bugs and violating object-oriented design principles.

To modernize such a system, **reverse engineering** can be applied by analyzing the existing code and redesigning it using **creational design patterns**, specifically the **Factory Method Pattern** or the **Abstract Factory Pattern**. These patterns encapsulate object creation and provide a centralized mechanism for creating objects, improving maintainability, scalability, flexibility, and code reuse.

1. Understanding the Legacy System

Assume a Hospital Management System supports three types of users:

- Doctor
- Patient
- Administrator

In the legacy implementation, object creation is performed directly inside business logic.

Legacy Code

```
if(userType.equals("Doctor"))                {
    user = new Doctor();
}
else if(userType.equals("Patient"))           {
    user = new Patient();
}
else if(userType.equals("Admin"))             {
    user = new Admin();
}
```

This code may appear in multiple modules, such as appointment scheduling, billing, and authentication.

Problems with the Legacy Design

1. **High Coupling:** Business logic depends directly on concrete classes.
2. **Code Duplication:** Similar object creation logic is repeated across the application.
3. **Poor Maintainability:** Adding a new class requires modifying many existing files.
4. **Limited Scalability:** The system becomes harder to extend as requirements grow.
5. **Testing Challenges:** Replacing objects with mock implementations is difficult.
6. **Violation of the Open/Closed Principle:** Existing code must be modified whenever a new user type is added.

2. Reverse Engineering Approach

Reverse engineering involves studying the existing system, identifying tightly coupled components, and redesigning them to improve software quality.

The redesign includes:

- Extracting object creation logic from business classes.
- Introducing interfaces or abstract classes.
- Creating factories responsible for object instantiation.
- Ensuring that client code depends on abstractions rather than concrete implementations.

3. Applying the Factory Method Pattern

The **Factory Method Pattern** defines a method for creating objects while allowing subclasses or factory classes to decide which concrete object to instantiate.

Step 1: Create a Common Interface

```
interface User {  
    void login();  
}
```

Step 2: Implement Concrete Classes

```
class Doctor implements User {  
    public void login() {  
        System.out.println("Doctor Login");  
    }  
}
```

```
class Patient implements User {  
    public void login() {  
        System.out.println("Patient Login");  
    }  
}
```

```
class Admin implements User {  
    public void login() {  
        System.out.println("Admin Login");  
    }  
}
```

Step 3: Create a Factory

```
class UserFactory {  
  
    public static User createUser(String type) {  
  
        if(type.equalsIgnoreCase("Doctor"))  
            return new Doctor();  
  
        if(type.equalsIgnoreCase("Patient"))  
            return new Patient();  
  
        if(type.equalsIgnoreCase("Admin"))  
            return new Admin();  
  
        return null;  
    }  
}
```

Step 4: Client Usage

```
User user = UserFactory.createUser("Doctor");  
user.login();
```

The client requests an object from the factory without knowing how it is created.

4. Applying the Abstract Factory Pattern

When multiple related object families exist, the **Abstract Factory Pattern** provides a more comprehensive solution.

For example, the Hospital Management System may need to create:

User Objects

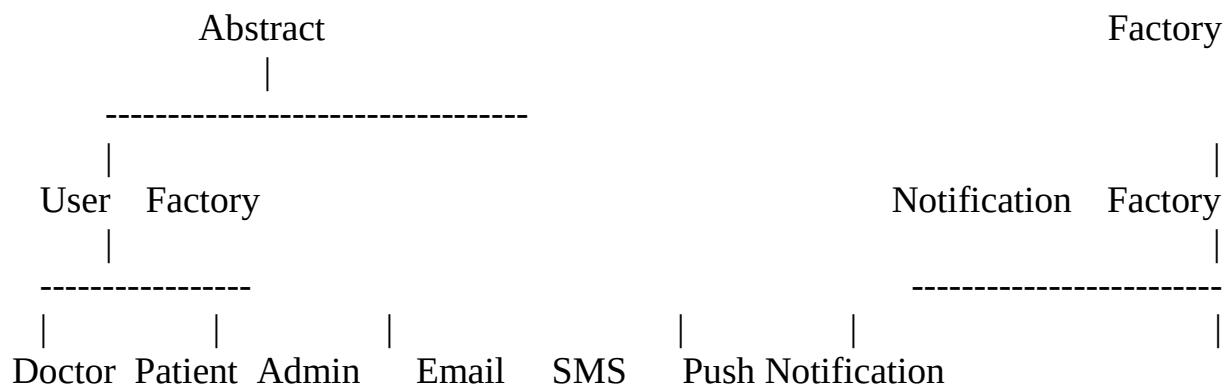
- Doctor
- Patient
- Admin

Notification Objects

- Email Notification
- SMS Notification
- Mobile Push Notification

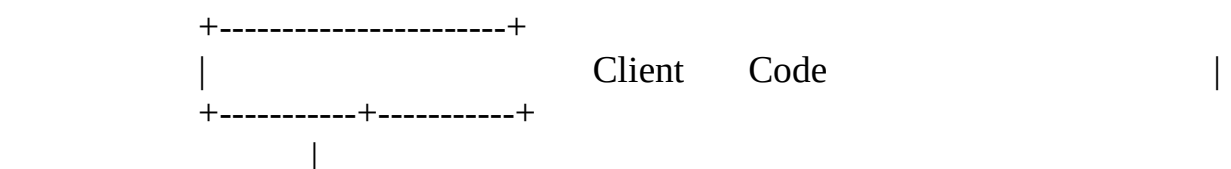
An abstract factory can create these related objects through specialized factories.

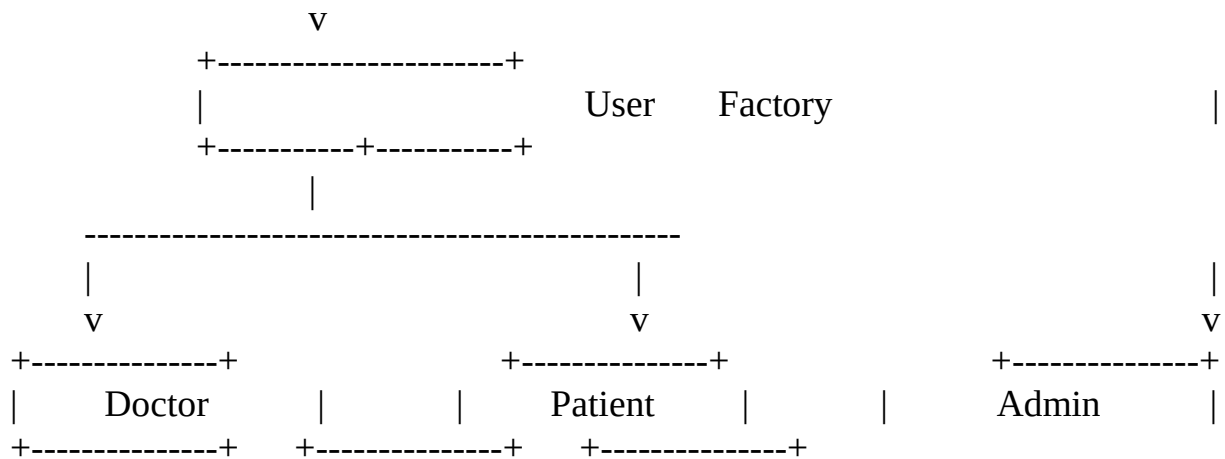
Conceptual Structure



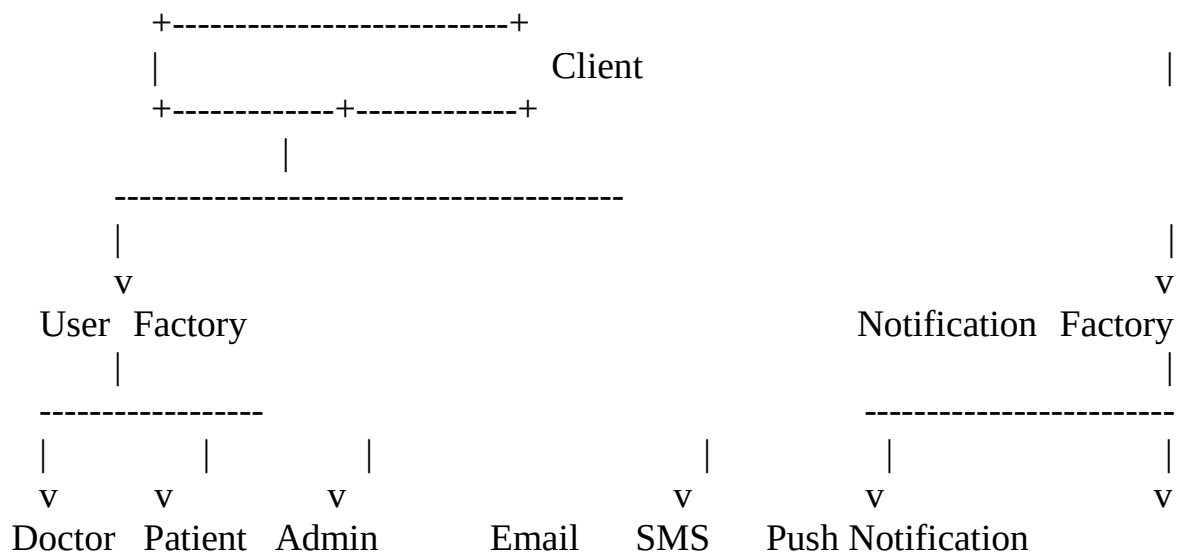
This approach keeps related object creation grouped together and simplifies future expansion.

5. Redesigned Architecture





For larger applications:



6. How the Redesign Improves Scalability

Scalability refers to the ease with which the software can accommodate new requirements.

Before Redesign

If a new user type, such as **Nurse**, is introduced:

- Every module containing object creation logic must be updated.
- Multiple files require modification.
- The chance of introducing errors increases.

After Redesign

Only the following changes are required:

1. Create a Nurse class implementing the User interface.
2. Add a single case in the UserFactory.

No changes are needed in client code or business logic.

Benefits

- Supports future expansion.
- Makes feature addition faster.
- Minimizes maintenance effort.

7. How the Redesign Improves Flexibility

Flexibility means adapting to changing requirements with minimal code changes.

The redesigned architecture provides flexibility by:

- Separating object creation from business operations.
- Allowing runtime selection of implementations.
- Enabling easy replacement of components.
- Supporting configuration-driven instantiation.

For example:

- A hospital may replace SMS notifications with WhatsApp notifications by modifying only the notification factory.
- Existing modules continue functioning without modification.

8. How the Redesign Improves Code Reusability

Object creation logic is centralized in factories instead of being duplicated.

Advantages

- Eliminates repeated code.
- Encourages reuse of common interfaces.
- Promotes modular programming.
- Simplifies maintenance.

For example, authentication, billing, scheduling, and reporting modules can all obtain User objects from the same factory instead of implementing separate creation logic.

9. Comparison Between Legacy and Redesigned System

Aspect	Legacy System	Redesigned System (Factory Pattern)
Object creation	Directly inside business classes	Centralized in factory classes
Coupling	High	Low
Code duplication	Significant	Minimal
Extensibility	Difficult	Easy
Maintainability	Low	High
Scalability	Poor	Excellent
Flexibility	Limited	High
Testing	Difficult	Easier due to abstraction
Reusability	Low	High

10. Real-World Example

Consider a hospital adding a new role called **Lab Technician**.

Legacy System

Developers must:

- Update appointment modules.
- Update billing modules.
- Update authentication modules.
- Update reporting modules.
- Modify every location where new objects are created.

Factory-Based System

Developers only:

1. Create a LabTechnician class.
2. Update the UserFactory.

All existing modules continue working without modification.

This demonstrates the practical benefits of centralized object creation.

11. Advantages of Using Factory Method and Abstract Factory

- Encapsulates object creation logic.
- Reduces dependencies between modules.
- Improves readability and organization.
- Supports the Open/Closed Principle.
- Makes testing easier by relying on interfaces.
- Facilitates future enhancements and maintenance.
- Promotes reuse of creation logic across the application.
- Enables seamless integration of new features and object families.

12. Conclusion

Reverse engineering the legacy system using **Factory Method** or **Abstract Factory** transforms a tightly coupled architecture into a modular and extensible design. By separating object creation from business logic, the system becomes easier to maintain, test, and expand. Centralized factories reduce code duplication, improve flexibility, and support the addition of new object types with minimal changes. Consequently, the redesigned system is significantly **more scalable, more adaptable to changing requirements, and more reusable**, making it better suited for long-term software evolution and enterprise-level applications.